

Busos AI — Ocena krytyczna, poprawki i słowniczek (wersja: Claude Fable 5)

Analiza plików `AI_PLAN.md` i `AI_research_potential.md`, wykonana od zera.

Pisane dla osoby, która buduje taki system **pierwszy raz** — każde narzędzie, każdy etap i każdy termin tłumaczę od podstaw.

Stan wiedzy: lipiec 2026, kluczowe fakty o narzędziach zweryfikowane z aktualną dokumentacją.

Struktura dokumentu

- **CZĘŚĆ I — Werdykt i krytyka.** Co w planie jest dobre, co jest błędem (w tym kilka błędów *w kodzie samego planu*, które zablokowałyby Cię na starcie), co poprawić i jakie narzędzia dobrać lepiej.
- **CZĘŚĆ II — Przewodnik etap po etapie.** Każda faza planu opisana po ludzku: co się w niej dzieje, jakie narzędzia w niej pracują i dlaczego, plus **słowniczek terminów tej fazy**.
- **CZĘŚĆ III — Słowniczek przekrojowy.** Terminy, które przewijają się przez cały projekt (podstawy ML, GIS, źródła danych, pojęcia z Twojego starego silnika, sprzęt).

Zasada nawigacji: nie rozumiesz słowa → Ctrl+F. Każdy termin użyty w Części I i II jest wyjaśniony w Części II (przy swojej fazie) albo w Części III.

CZĘŚĆ I — Werdykt i krytyka

1. Werdykt ogólny

`AI_PLAN.md` to **dobry, dojrzały plan inżynierski** — zdecydowanie lepszy niż `AI_research_potential.md`, z którego wyrósł. Jego trzy największe zalety:

1. **Rygor "baseline first".** Plan każe najpierw zbudować prosty model tabelaryczny (XGBoost) i dopiero potem sprawdzać, czy sieć grafowa (GNN) daje cokolwiek ponad to. To jest zgodne z aktualnym stanem nauki: badania z lat 2022–2026 konsekwentnie pokazują, że na płaskich danych tabelarycznych modele drzewiaste wciąż zwykle wygrywają z sieciami neuronowymi. Plan sam to przyznaje (przywołując Geerts et al. 2025) i definiuje wartość GNN jako *wgląd przestrzenny*, nie jako lepsze MAE. To bardzo dojrzałe.
2. **Uczciwość wobec wyników negatywnych.** "Jeśli GNN nie pobije baseline'u — udokumentuj dlaczego" to postawa badawcza, która w portfolio wygląda lepiej niż naciągnięty sukces.
3. **Appendix A (korekty do dokumentu badawczego).** Plan systematycznie prostuje halucynacje i przesadę z `AI_research_potential.md`. To wzorcowa praktyka pracy z materiałami generowanymi przez AI.

Jednocześnie plan zawiera **kilka konkretnych błędów technicznych i metodologicznych**, które opisuję niżej. Żaden nie unieważnia całości — ale dwa z nich (2.1 i 2.2) zatrzymałyby Cię w pierwszym tygodniu, a jeden (2.3) sprawiłby, że Twój flagowy wynik ("TOD Premium w PLN za kurs") byłby policzony **niepoprawnie**, mimo że kod by działał.

2. Błędy do naprawienia (od najpoważniejszych)

2.1 Kod GNN w planie nie zadziała na grafie, który plan każe zbudować

To najpoważniejsza niespójność wewnętrzna planu. Zestawmy trzy sekcje:

- **§4.2** buduje graf **heterogeniczny** (`HeteroData`): węzły typu `stop`, `poi`, `transaction`; etykieta (`cena_y`) siedzi na węzłach typu `transaction`.
- **§5.1** definiuje model **homogeniczny**: `GATv2Conv(in_channels, ...)` z forwardem (`x`, `edge_index`, `edge_attr`) — czyli model dla grafu z *jednym* typem węzłów. Importuje `to_hetero`, ale nigdy go nie używa. `training_step` czyta `batch.train_mask` i `batch.y` tak, jakby etykiety były na tych samych węzłach, które wchodziły do modelu.
- **§5.2** ustawia `NeighborLoader(..., input_nodes='stop')` — czyli próbkowanie startuje od przystanków, podczas gdy cel (`y`) wisi na węzłach `transaction`.

Efekt: gdybyś przepisał ten kod 1:1, dostałbyś błędy typów/kształtów albo — gorzej — kod, który "jakoś" ruszy po drobnych poprawkach i będzie trenował nie na tym, na czym myślisz. **Musisz podjąć jedną decyzję architektoniczną, zanim napiszesz linijkę Fazy 4:**

- **Opcja prosta (rekomendowana na MVP):** przenieś cel na węzły `stop`. Czyli tak jak w baseline: każdy przystanek ma jedną etykiętę `rcn_median_price_m2`. Graf: `stop` i `poi` jako typy węzłów, model opakowany w `to_hetero(model, data.metadata())`, `NeighborLoader` z `input_nodes=('stop', train_mask)`. Wszystko się domyka i jest wprost porównywalne z XGBoostem (ta sama etykieta, ten sam podział).
- **Opcja ambitna (na później):** przewiduj cenę **na poziomie pojedynczej transakcji** (węzły `transaction` z krawędziami do pobliskich przystanków). Metodologicznie silniejsza (patrz 2.4), ale wymaga przepisania loadera, masek i porównania z baselinem, który też musiałby zejść na poziom transakcji.

Nie wybieraj obu naraz — plan obecnie nieświadomie miesza jedną z drugą.

2.2 Instalacja PyTorch Geometric na ROCm — plan jest przestarzały w złą stronę

Plan (§1.2) twierdzi, że nie istnieją gotowe paczki (wheels) ROCm dla rozszerzeń PyG i budżetuje 1–2 dni na kompilację ze źródeł. Dwie korekty:

1. Istnieje zewnętrzne repozytorium `pyg-rocm-build` z gotowymi wheelsami rozszerzeń PyG pod ROCm — oficjalna dokumentacja PyG wprost do niego odsyła. Zanim cokolwiek skompilujesz, sprawdź, czy jest tam kombinacja pod Twój PyTorch + ROCm.
2. Ważniejsze: **sprawdź, czy w ogóle potrzebujesz tych rozszerzeń.** Nowoczesny PyG (2.5+) ma wbudowane natywne odpowiedniki dawnych `torch-scatter` / `torch-sparse` i dla typowego treningu GATv2 + `NeighborLoader` samo `pip install torch-geometric` na torchu ROCm często wystarcza. Rozszerzenia (`pyg-lib` itd.) przyspieszają próbkowanie, ale przy grafie wielkości Kielc różnica będzie kosmetyczna.

Poprawiona kolejność: (a) sam `torch-geometric`, test dymny treningu → (b) jeśli próbkowanie za wolne: wheels z `pyg-rocm-build` → (c) dopiero w ostateczności build ze źródeł / Docker `rocm/pytorch`.
Oszczędność: realnie 1–2 dni i sporo frustracji.

2.3 Sposób liczenia "TOD Premium" z SHAP jest metodologicznie błędny

To subtelne, ale kluczowe, bo dotyczy Twojej flagowej liczby. Plan (§3.3) robi:

```
tod_premium_per_kurs = np.median(shap_values[:,  
    'transit_freq'].values)
```

i nazywa wynik "ile PLN/m² dodaje +1 kurs/h". **To nie jest to samo.** Wartość SHAP dla cechy `transit_freq` u obserwacji *i* mówi: "o ile *obecny poziom* częstotliwości w tym miejscu odchyła predykcję od średniej predykcji". Mediana tych wartości po zbiorze testowym to "typowy wkład poziomu częstotliwości do ceny" — a nie "efekt

krańcowy dodania jednego kursu". Te dwie wielkości mogą się różnić i znakiem, i rzędem wielkości.

Jak policzyć to, co naprawdę chcesz powiedzieć (" +1 kurs/h → ile PLN/m²):

- **Wariant kontrfaktyczny (najprostszy i najuczciwszy):** weź zbiór testowy, zrób kopię z `transit_freq += 1`, policz `model.predict(X_plus) - model.predict(X)`, uśrednij. To jest wprost "przewidywana zmiana ceny przy +1 kursie, przy pozostałych cechach bez zmian".
- **Wariant wykresowy:** nachylenie **wykresu zależności SHAP** (SHAP dependence plot) dla `transit_freq` w interesującym zakresie, albo klasyczne **PDP/ALE** (partial dependence / accumulated local effects, są w `sklearn.inspection`). ALE jest odporniejsze, gdy cechy są skorelowane — a Twoje są (częstotliwość koreluje z gęstością POI).

Do obu wariantów dopisz w raporcie jedno zdanie zastrzeżenia: to **efekt predykcyjny modelu**, nie efekt przyczynowy interwencji (patrz 2.6). Mediana SHAP-ów może zostać w raporcie — ale pod nazwą "typowy wkład częstotliwości do wyceny", nie "premium za kurs".

2.4 Cel na poziomie przystanku = ciche podwójne liczenie i zawyżone metryki

Etykieta `rcn_median_price_m2` to mediana transakcji **w promieniu 500 m od przystanku**. Przystanki w centrum stoją co 200–300 m, więc ich bufor 500 m **mocno się nakładają** — te same transakcje wchodzą do etykiet wielu sąsiednich przystanków. Konsekwencje:

- **Pseudoreplikacja:** dane wyglądają na liczniejsze i "gładzsze", niż są. R² i MAE wychodzą lepsze, niż zasługuje model.
- **Przeciek mimo podziału blokowego:** podział po heksach H3-7 pomaga, ale przystanki po dwóch stronach granicy bloku wciąż mogą dzielić te same transakcje w buforach. Trening "widzi" fragment odpowiedzi z testu.

Co zrobić (wybierz jedno, wystarczy):

1. **Minimalnie:** przy podziale przestrzennym odrzuć ze zbioru testowego przystanki, których bufor przecina bufor jakiegokolwiek przystanku treningowego (strefa buforowa/"spatial buffer" między train a test). Kilka linii kodu, uczciwe metryki.
2. **Lepiej:** modeluj na poziomie transakcji (cechy = cechy najbliższego przystanku / otoczenia transakcji, etykieta = cena tej jednej transakcji). Znika podwójne liczenie, rośnie liczba próbek (9 550 zamiast ~kilkuset), a mediana per przystanek zostaje jako produkt raportowy, nie jako cel uczenia.

Dodatkowo: ustal **minimalny próg liczby transakcji** na przystanek (np. ≥ 5 w buforze). Mediana z 2 transakcji to szum, nie sygnał — a plan obecnie takich progów nie stawia.

2.5 API `city2graph` w planie jest zmyślone

Kod `from city2graph import Graph, Morphology;`
`Graph.gdf_to_pyg(...)` (§4.2, oraz kryterium akceptacji Fazy 0) **nie istnieje w tej bibliotece**. Realne API to funkcje na poziomie modułu: `city2graph.morphological_graph()`, `contiguity_graph()`, `group_nodes()`, `gdf_to_pyg(nodes_dict, edges_dict)` (zwraca `HeteroData`), a GTFS ładuje się przez `load_gtfs()` (zwraca połączenie DuckDB). Gdybyś wkleił kod z planu, dostałbyś `ImportError` w pierwszej godzinie i mógłbyś stracić pół dnia na szukanie winy u siebie. Ogólna zasada: **kod w planach pisanych przez AI to pseudokod ilustrujący intencję** — przed implementacją każdy przykład konfrontuj z żywą dokumentacją (tu: city2graph.net).

2.6 Fazy 6–7: pomieszane nazewnictwo modeli i za mocny język przyczynowy

Dwa osobne problemy:

(a) **"Deep Gravity", które nie jest Deep.** §7.1 w nagłówku obiecuje "Deep Gravity" (neuronowy model przeptywów), ale kod używa `skmob.models.gravity.Gravity` — czyli **klasycznego, statystycznego modelu grawitacyjnego** z lat 40., tylko dopasowanego do danych. To zupełnie inne narzędzia. Klasyczna

grawitacja na MVP jest zresztą *lepszym* wyborem (prostsza, tłumaczalna, nie wymaga GPU) — po prostu nazwij ją po imieniu, bo recenzent, który zna oba, wychwyci rozjazd w 10 sekund. Sprawdź też higienę samej paczki: `scikit-mobility` jest rozwijana wolniej niż reszta Twojego stacku — przetestuj instalację z Twoją wersją Pythona/NumPy, zanim zaplanujesz na niej fazę.

(b) Korelacja ≠ przyczynowość. Faza 7 symuluje "wstaw przystanek → GNN przewiduje wzrost cen". Model uczył się **współwystępowania** cech z cenami, nie skutków interwencji. Przystanki nie powstają losowo — stawia się je tam, gdzie już jest gęsto i drogo, więc model może po prostu odtwarzać tę politykę lokalizacyjną. Wniosek "dodanie przystanku podniesie ceny o X" jest z tych danych **niewyprowadzalny**. Rozwiązanie kosztuje zero pracy i ratuje wiarygodność: konsekwentnie pisz "obszar o profilu współwystępującym z wyższymi cenami — kandydat do analizy", oznacz Fazy 6–7 jako *eksploracyjne* i dopisz to zastrzeżenie do GeoJSON-ów, które pójdą na dashboard.

2.7 r5py: nie karm silnika całą Polską

§4.1 podaje `osm_pbf="data/poland/poland-latest.osm.pbf"` do zbudowania sieci dla... Kielc. Silnik R5 zbuduje wtedy graf ulic **całego kraju** (RAM, czas, dysk), z którego użyjesz ułamka procenta. Wytnij najpierw okolicę Kielc (np. `osmium extract` z poligonem strefy — masz to już opanowane w kroku 03–05 swojego pipeline'u!) i podaj mały PBF. To zamienia ryzyko "r5py zjada 32 GB RAM" (które plan wpisuje do rejestru ryzyk) w nie-problem.

2.8 Bramka decyzyjna na Moranie: progi z sufitu

§3.4 ustala: Moran's I reszt $> 0,15 \rightarrow$ GNN uzasadniony; $< 0,05 \rightarrow$ nie. Te liczby są arbitralne i zależne od wyboru wag przestrzennych (dlaczego KNN z $k=8$, a nie $k=5$ albo odległościowe?). Poprawka tania i elegancka: licz **istotność permutacyjną** (`Moran(..., permutations=999)`, patrz `mi.p_sim` w `esda`) i decyzję opieraj na "I istotne przy $p < 0,01$ i praktycznie duże", a wrażliwość na k pokaż dla 2–3 wartości. Jedna komórka notebooka, a metodologia z "liczb magicznych" robi się obroniona.

2.9 Harmonogram: 30 dni to tempo osoby, która robiła to wielokrotnie

Gantt sumuje się do ~6 tygodni przy założeniu, że nic nie zaskoczy i wszystko robisz drugi raz w życiu. Robisz pierwszy raz — realnie licz **8–12 tygodni** na Fazy 0–5 z nauką po drodze. To nie zarzut, tylko kalibracja oczekiwań: lepiej zaplanować wolniej i dowieźć, niż ścigać się z fikcyjnym terminem. Dobra wiadomość: Fazy 0–2 (środowisko + eksport + baseline) są niezależne od całej reszty i dają publikowalny wynik same w sobie — to Twój naturalny "kamień milowy nr 1".

3. Rzeczy przewymiarowane na MVP (świadomie odłóż)

Komponent	Dlaczego na wyrost dla samych Kielc	Co zamiast
Qdrant (Faza 8)	Baza wektorowa błyszczy przy setkach tysięcy wektorów; Kielce to setki przystanków	<code>sklearn.neighbors.NearestNeighbors</code> na macierzy embeddingów — 5 linii; Qdrant dołóż przy skali krajowej
PyTorch Lightning	Dodatkowa warstwa abstrakcji utrudnia debugowanie pierwszemu GNN-owi	Czysta pętla treningowa w PyTorch (~25 linii) — zobaczysz każdy krok i więcej się nauczysz

Komponent	Dlaczego na wzrost dla samych Kielc	Co zamiast
FastAPI (Faza 8)	Dashboard może czytać statyczne GeoJSON-y z dysku	Eksport plików; API zbuduj, gdy pojawią się zapytania "na żywo"
Modele z §11.1 (VGAE, GraphSAGE, ensemble)	Każdy to tydzień+; rozmywają MVP	Zostaw w "co dalej"; z listy najtańszy realny dodatek to GWR (patrz niżej)

Plan sam deklaruje zasadę minimalizmu — te cztery cięcia to po prostu
twardsze trzymanie się jego własnych reguł.

4. Lepsze / brakujące narzędzia

Dodaj do Fazy 2 (tanio, duży zysk):

- **LightGBM** obok XGBoost i CatBoost. Zwykle najszybszy z trójki, świetny na w pełni numerycznych cechach jak Twoje. Tabela porównawcza trzech GBM-ów to mocny sygnał rzetelności w portfolio. (Uwaga: główny atut CatBoosta — obsługa cech kategorycznych — u Ciebie prawie nie gra, bo wszystkie cechy są liczbowe; traktuj go czysto jako sanity check, tak jak plan pisze.)
- **Optuna** do strojenia hiperparametrów zamiast ręcznego zgadywania `max_depth` / `learning_rate`. Kilkanaście linii, a baseline potrafi zauważalnie podskoczyć — co jest ważne, bo *uczciwie mocny baseline* to sedno Twojej metodologii.
- **PDP/ALE z `sklearn.inspection`** — do poprawnego policzenia efektu "+1 kurs/h" (patrz 2.3).

Rozważ jako tani "pomost przestrzenny" między Fazą 2 a 4:

- **GWR (Geographically Weighted Regression, paczka `mgwr`)** — plan wymienia ją tylko w "portfolio amplifiers", a to najtańszy sposób, by pokazać przestrzenną niestacjonarność (czy premium za transport różni się między dzielnicami) **bez** czekania na GNN. Dzień pracy, klasyczna, obroniona metoda, ładna mapa do dashboardu.

Miej na radarze (nie do MVP):

- **Overture Maps** jako czystsze źródło budynków, gdy OSM dla Kielc okaże się dziurawy — `city2graph` wspiera je natywnie.
- **TabPFN / tabelaryczne modele foundation (2025–2026)** — ciekawe przy małych zbiorach, ale to eksperyment do sekcji "future work".

Słusznie odrzucone przez plan (potwierdzam): Neo4j (PyG + Parquet wystarczą przy treningu jednomaszynowym), PyG Temporal (32 GB RAM to za mało na grafy czasowe), Reinforcement Learning z `AI_research_potential.md` (to osobny, wielomiesięczny projekt).

5. Ostrzeżenie o `AI_research_potential.md`

Ten dokument jest **inspiracją kierunków, nie źródłem faktów**. Zawiera stwierdzenia brzmiące bardzo konkretnie, które są nieaktualne lub zmyślane — część plan już skorygował w Appendixie A. Dodatkowe czerwone flagi: przywoływane publikacje ("BD-GNN, MDPI 2025", "A-SRGCNN 2025", "GNN-SWIM", "GNNUI") **zweryfikuj indywidualnie**, zanim którąkolwiek zacytujesz w blogu czy portfolio — modele językowe notorycznie wymyślają wiarygodnie brzmiące tytuły prac. Heurystyka: im bardziej konkretna liczba, wersja lub nazwa w tym pliku, tym mocniej ją sprawdź.

6. Podsumowanie werdyktu

Plan jest **gotowy do realizacji po jednym dniu poprawek**:

1. Rozstrzygnij architekturę celu w GNN (etykieta na `stop` — rekomendowane) i dopisz `to_hetero` — **inaczej Faza 4 nie ruszy** (2.1).
2. Zmień recepturę TOD Premium z "mediana SHAP" na kontryfaktyk +1 / ALE — **inaczej flagowa liczba będzie zła** (2.3).
3. Dodaj próg min. transakcji i strefę buforową w podziale train/test (2.4).
4. Popraw sekcję instalacji PyG (wheels ROCm / brak potrzeby rozszerzeń) i przykłady `city2graph` (2.2, 2.5).
5. Przemianuj "Deep Gravity" na model grawitacyjny, wytnij PBF do Kielc, oznacz Fazy 6–7 jako eksploracyjne (2.6, 2.7).

Rdzeń (Fazy 0–5) jest solidny. Fazy 6–8 to "nice to have" — dowiedz najpierw baseline i GNN z uczciwym porównaniem, bo to jest teza całego projektu.

CZĘŚĆ II — Przewodnik etap po etapie (dla totalnego laika)

Każda faza: **co się dzieje** → **narzędzia i ich role** → **słowniczek tej fazy**.
Czytaj po kolei albo skacz do fazy, przy której właśnie pracujesz.

FAZA 0 — Przygotowanie środowiska ("budowa warsztatu")

Co się dzieje: zanim cokolwiek policzysz, komputer musi umieć trzy rzeczy: (1) używać karty graficznej do obliczeń AI, (2) rozumieć grafy, (3) liczyć realne czasy dojazdu po mieście. Ta faza to instalacja i sprawdzenie, że każdy klocek działa.

Narzędzia i ich role:

- **ROCm** — oprogramowanie AMD, dzięki któremu Twoja karta Radeon może liczyć sieci neuronowe. To odpowiednik CUDA znanego z kart NVIDIA. Bez niego karta jest do AI bezużyteczna, a trening szedłby na procesorze wielokrotnie wolniej. Rola: fundament, na którym stoi wszystko dalej.
- **PyTorch** — najpopularniejszy "silnik" do budowy i trenowania sieci neuronowych. Ty opisujesz kształt modelu, PyTorch sam liczy, jak poprawiać jego parametry, żeby błąd malał. Rola: rdzeń całej części AI.
- **PyTorch Geometric (PyG)** — rozszerzenie PyTorch do sieci działających **na grafach** (węzły + połączenia), zamiast na zwykłych tabelach. Rola: w nim zbudujesz i wytrenujesz GNN.
- **city2graph** — "klej" między danymi mapowymi (GeoPandas, OSM, GTFS) a formatem grafowym PyG. Zamienia Twoje warstwy geograficzne w graf gotowy do "połknięcia" przez sieć. Rola: oszczędza żmudne, podatne na błędy ręczne budowanie grafu. (Pamiętaj o poprawce 2.5 — API bierz z dokumentacji, nie z planu.)
- **r5py + Java (JDK 21)** — biblioteka licząca **realne czasy dojścia** po chodnikach i z rozkładami jazdy. Pod spodem działa silnik R5 napisany w Javie — stąd wymóg zainstalowania Javy w wersji 21+. Rola: dostarcza "prawdziwe" odległości, które staną się połączeniami w grafie.
- **conda / pip** — dwa menedżery pakietów Pythona (programy instalujące biblioteki). Conda bywa wygodniejsza przy trudnych zależnościach (r5py + Java), pip jest standardem dla reszty.
- **Docker** — awaryjny "kontener": gotowe, zamknięte środowisko (`rocm/pytorch`), którego użyjesz, gdyby instalacja na żywym systemie sprawiała problemy.

Słowniczek Fazy 0:

- **GPU / VRAM** — karta graficzna / jej własna pamięć (u Ciebie 16 GB). Trening dzieje się na GPU; VRAM to twardy limit tego, ile naraz zmieści się "na karcie".
- **CUDA** — technologia NVIDIA do obliczeń na GPU. Mylące, ale ważne: w PyTorchu funkcja `torch.cuda.is_available()` zwraca

`True` także dla kart AMD przez ROCm — nazwa została z historii.

- **RDNA 4 / gfx1200** — architektura / techniczny kryptonim Twojego Radeona RX 9060 XT. ROCm musi go rozpoznawać, stąd test `rocm_info | grep gfx1200`.
- **wheel (.whl)** — gotowa, prekompilowana paczka Pythona: instaluje się w sekundy. Przeciwnieństwo: **build ze źródeł** — samodzielna kompilacja kodu, powolna i kapryśna. Zawsze najpierw szukaj wheela.
- **venv / środowisko wirtualne** — izolowany "worek" na biblioteki jednego projektu, żeby wersje nie gryzły się z systemem i innymi projektami.
- **Test dymny (smoke test)** — najprostsze możliwe sprawdzenie "czy w ogóle działa" (np. `torch.cuda.is_available()`), zanim zainwestujesz czas w rzeczy skomplikowane.
- **Kryteria akceptacji** — checklista "co musi być spełnione, żeby uznać fazę za skończoną". Plan ma je dla każdej fazy — trzymaj się ich, chronią przed rozgrzebaniem wszystkiego naraz.

FAZA 1 — Przygotowanie danych ("surowiec bez przypraw")

Co się dzieje: Twój stary silnik (kroki 00–15) produkuje dane już "doprawione" ludzkimi założeniami: wagami tierów, modelem Huffa, Z-Score'ami. AI ma odkryć wagi **samo** — więc budujesz nowy skrypt 15b, który eksportuje te same dane, ale **surowe**: gołe liczniki ("ile przychodzi w 500 m"), gołe dystanse ("ile metrów do najbliższej szkoły"), gołą częstotliwość z rozkładów. Dodatkowo oznaczasz flagą rekordy, które kiedyś naprawiały skrypty 08/09 — żeby modele mogły je opcjonalnie wykluczyć.

Narzędzia i ich role:

- **GeoPandas** — Pandas (tabele w Pythonie) rozszerzony o geometrię. Podstawowe narzędzie do "policz, co jest w promieniu X od punktu". Rola: całe liczenie cech przestrzennych.

- **H3 (Uber H3)** — system dzielenia mapy na równe **sześciokąty** w różnych rozdzielczościach. Rola tutaj: każdemu przystankowi przypisujesz indeks hekza (`h3_res9`), co później posłuży do agregacji i do uczciwego podziału danych.
- **Parquet** — bardzo szybki, kolumnowy format plików z tabelami. Rola: format wyjściowy macierzy cech (`raw_features_ai.parquet`), który błyskawicznie wczytasz w kolejnych fazach.

Słowniczek Fazy 1:

- **Cecha (feature, X)** — pojedyncza informacja wejściowa o obiekcie, np. `transit_freq` (kursy/h) albo `poi_count_health` (liczba placówek zdrowia w 500 m). Model dostaje zestaw cech i na ich podstawie zgaduje odpowiedź.
- **Zmienna celu (target, Y)** — to, co model ma przewidzieć: u Ciebie cena za metr (`rcn_median_price_m2`).
- **Macierz cech (feature matrix)** — tabela: wiersze = obiekty (przystanki), kolumny = cechy. Podstawowe "paliwo" każdego modelu.
- **Schema (schemat)** — spis kolumn z typami i opisem. Tabela w §2.2 planu to właśnie schemat — trzymaj się go, to Twój kontrakt między fazami.
- **Feature engineering** — ręczne wymyślanie i liczenie cech. Twój stary silnik to feature engineering w skrajnej formie (wagi wpisane na sztywno); AI ma go częściowo zastąpić uczeniem z danych.
- **Flaga naprawy (`repair_source` , `repair_confidence`)** — znacznik "ten rekord był kiedyś uszkodzony i został naprawiony algorytmem". Pozwala trenować w trybie ścisłym (tylko czyste rekordy) albo szerokim (z naprawionymi, świadomie akceptując szum).
- **Nullable / null** — kolumna, która może być pusta. `repair_source = null` znaczy "rekord czysty, nigdy nie naprawiany".
- **GPKG (GeoPackage)** — plik bazy danych przestrzennej (punkty, poligony + atrybuty). Format Twoich warstw wejściowych (`stops.gpkg` itd.).

- **Mediana** — wartość środkowa (połowa obserwacji poniżej, połowa powyżej). Odporniejsza na wartości absurdalne niż średnia — dlatego cena per przystanek to mediana, nie średnia.
 - **Katchment / bufor 500 m** — umowny "obszar oddziaływania" przystanku, z którego zbierasz transakcje i POI.
-

FAZA 2 — Baseline: XGBoost ("poprzeczka, którą trzeba przeskoczyć")

Co się dzieje: trenujesz prosty, mocny model tabelaryczny, który z cech przystanku przewiduje cenę. Mierzysz, jak dobrze mu idzie (MAE, R^2), wyciągasz z niego interpretację (SHAP: co napędza cenę) i sprawdzasz Moranem, czy w jego błędach została struktura przestrzenna. Wynik tej fazy to punkt odniesienia — wszystko, co zbudujesz później, będzie porównywane do niego.

Narzędzia i ich role:

- **XGBoost / LightGBM / CatBoost** — trzy biblioteki tej samej rodziny: **gradient boosting na drzewach decyzyjnych**. Budują setki małych drzewek, z których każde kolejne poprawia błędy poprzednich. Na danych tabelarycznych to zwykle najskuteczniejsza klasa modeli w ogóle. Rola: model bazowy (i, uczciwie mówiąc, faworyt całego wyścigu).
- **SHAP** — metoda tłumaczenia predykcji: rozkłada każdą prognozę na wkłady poszczególnych cech ("ta cena jest wysoka, bo +900 zł z częstotliwości, +400 zł z handlu, -200 zł z odległości do centrum"). Oparta na teorii gier (wartości Shapleya). Rola: interpretacja baseline'u. **Pamiętaj o poprawce 2.3** — do liczby "za +1 kurs/h" użyj kontrfaktyku/ALE, nie mediany SHAP.
- **Optuna** (*rekomendowany dodatek*) — automat do strojenia hiperparametrów: sam inteligentnie przeszukuje kombinacje ustawień zamiast Twojego zgadywania.
- **esda + libpysal** — biblioteki statystyki przestrzennej. Rola: policzenie **Morana I** na resztach modelu — czyli testu "czy błędy są

skupione geograficznie" (jeśli tak, to znaczy, że model przegapia coś przestrzennego i GNN ma czego szukać).

Słowniczek Fazy 2:

- **Regresja** — zadanie, w którym przewidujesz liczbę (cenę), a nie kategorię.
- **Uczenie nadzorowane (supervised)** — uczenie z "prawidłowymi odpowiedziami" (prawdziwe ceny z RCN); model uczy się je odtwarzać.
- **Drzewo decyzyjne** — model-"ankieta": sekwencja pytań ("częstotliwość > 6? dystans do centrum < 800 m?") prowadząca do prognozy. Pojedyncze drzewo jest słabe; tysiąc drzew poprawiających się nawzajem (boosting) — bardzo mocne.
- **Gradient boosting** — technika: każde kolejne drzewo uczy się na **błędach** sumy poprzednich. Stąd nazwy XGBoost ("extreme gradient boosting") itd.
- **Zbiór treningowy / testowy (train/test split)** — dane dzielisz na część do nauki (np. 80%) i część, której model nigdy nie widział (20%), na której uczciwie mierzysz jakość.
- **Przestrzenny podział blokowy (spatial block split)** — zamiast losować pojedyncze punkty, losujesz całe **obszary** (heksy H3-7) do train albo do test. Chroni przed sytuacją, w której punkt testowy leży 50 m od treningowego i model "podgląda" odpowiedź przez sąsiedztwo.
- **Wyciek danych (leakage) / wyciek przestrzenny (spatial leakage)** — każda sytuacja, w której do treningu przecieka informacja z testu (albo z samej odpowiedzi). Objaw: bajeczne wyniki na teście, kompromitacja na nowych danych. Patrz też poprawka 2.4 o nakładających się buforach.
- **Pseudoreplikacja** — traktowanie tych samych obserwacji policzonych wielokrotnie jako niezależnych danych. Zawyża pewność siebie modelu i metryki.
- **MAE (Mean Absolute Error)** — "o ile złotych średnio się myłę". $MAE = 500$ znaczy: typowa pomyłka to 500 zł/m². Im mniej, tym lepiej; łatwe do wytłumaczenia laikowi.

- **R^2 (współczynnik determinacji)** — jaki ułamek zmienności cen model wyjaśnia: 0 = nic (zgadywanie średniej), 1 = idealnie. Plan celuje w $R^2 > 0,5$ na teście.
- **Hiperparametr** — ustawienie modelu wybierane **przed** treningiem (głębokość drzew `max_depth`, tempo uczenia `learning_rate`, liczba drzew `n_estimators` ...). W odróżnieniu od parametrów, których model uczy się sam.
- **Regularyzacja (`reg_alpha`, `reg_lambda`, `subsample`)** — techniki "hamowania" modelu, żeby nie wykuł danych na pamięć (patrz overfitting w Części III).
- **Early stopping** — przerwanie treningu, gdy jakość na zbiorze walidacyjnym przestaje się poprawiać przez N rund (u Ciebie 20). Chroni przed przeuczeniem i oszczędza czas.
- **Reszty (residuals)** — różnice prawda – predykcja dla każdego punktu. Ich analiza mówi, **gdzie i jak** model się myli.
- **Moran's I** — miara skupienia przestrzennego (od ok. -1 do ok. +1). Liczona na resztach: wysoka i istotna = błędy tworzą geograficzne "plamy" = model przegapia przestrzeń. To Twoja bramka decyzyjna przed GNN (z poprawką 2.8: patrz na istotność permutacyjną, nie tylko na próg).
- **Wagi przestrzenne / KNN (k najbliższych sąsiadów)** — definicja "kto z kim sąsiaduje" potrzebna Moranowi; `KNN k=8` = każdy punkt sąsiaduje ze swoimi 8 najbliższymi.
- **PDP / ALE (partial dependence / accumulated local effects)** — wykresy pokazujące, jak predykcja zmienia się przy przesuwaniu **jednej** cechy. To jest właściwe narzędzie do liczby "+1 kurs/h → ile PLN" (ALE lepsze przy cechach skorelowanych).
- **Sanity check** — szybki test "czy wynik w ogóle ma sens" (tu: CatBoost jako drugi model — jeśli dwa różne modele dają podobny wynik, ufasz mu bardziej).

FAZA 3 — Budowa grafu ("miasto jako sieć, nie tabela")

Co się dzieje: zamieniasz Kielce w graf: przystanki i POI to **węzły**, a realne dojścia piesze między nimi to **krawędzie** z wagą "ile minut". Kluczowa zmiana filozoficzna: odległość w linii prostej idzie do kosza — liczy się, jak człowiek naprawdę dojdzie (rzeka bez mostu = brak krawędzi albo krawędź bardzo droga).

Narzędzia i ich role:

- **r5py** — liczy macierz czasów podróży (każdy przystanek → każdy POI, pieszo, z limitem 20 min). Rola: dostarcza wagi krawędzi. Pamiętaj o poprawce 2.7 (wytnij PBF do Kielc) i o limicie pamięci `set_max_memory("12G")`.
- **OSMnx** — buduje graf ulic/chodników z danych OpenStreetMap. Rola pomocnicza: topologia sieci pieszej, a w Fazie 7 "przyklejanie" kandydatów na przystanki do najbliższej drogi.
- **city2graph / ręczna konstrukcja PyG** — składa węzły + krawędzie + cechy w obiekt `HeteroData`, który PyG rozumie. Plan słusznie ma plan B (ręczna konstrukcja), gdyby city2graph nie współpracował z polskimi danymi.

Słowniczek Fazy 3:

- **Graf** — struktura z **węzłów** (nodes: przystanki, POI) i **krawędzi** (edges: połączenia między nimi).
- **Krawędź skierowana / nieskierowana** — połączenie z kierunkiem ("stop → poi") lub bez. Dojście piesze jest w praktyce symetryczne, ale w `HeteroData` typ krawędzi ma kierunek w nazwie.
- **Cecha krawędzi (`edge_attr`)** — liczby przypisane połączeniu, u Ciebie czas przejścia w minutach. To przez nie graf "wie", że 4 minuty do szkoły to co innego niż 18.
- **`edge_index`** — techniczny zapis krawędzi w PyG: tablica $2 \times E$ par (skąd, dokąd), gdzie E = liczba krawędzi.
- **Graf heterogeniczny (`HeteroData`)** — graf z **różnymi typami** węzłów i krawędzi (typ "stop", typ "poi", relacja "walks_to"). Twój graf jest heterogeniczny, bo przystanek i sklep to obiekty o różnych zestawach cech.

- **Odległość euklidesowa** — w linii prostej, "jak lata ptak". Szybka do policzenia, w mieście często kłamie (mury, rzeki, tory).
- **Odległość sieciowa (network distance)** — po realnych chodnikach/ulicach, "jak chodzi człowiek". Wolniejsza do policzenia, prawdziwa. Sedno Fazy 3.
- **Macierz czasów podróży (Travel Time Matrix, TTM)** — wielka tabela "z każdego do każdego: ile minut". Główny produkt r5py.
- **PBF (.osm.pbf)** — skompresowany plik z danymi OpenStreetMap; wsad dla r5py i OSMnx.
- **Węzeł izolowany (isolated node)** — węzeł bez żadnej krawędzi. GNN nie ma skąd zebrać o nim informacji — plan słusznie każe sprawdzić, że takich nie ma.
- **Topologia** — "kształt połączeń" sieci: co z czym się łączy, niezależnie od dokładnych współrzędnych.
- **Okno odjazdów (departure_time_window)** — r5py symuluje podróże w przedziale czasu (np. poniedziałek 8:00–10:00) i uśrednia, żeby wynik nie zależał od jednej pechowej minuty rozkładu.

FAZA 4 — Trening GNN ("model, który patrzy na sąsiadów")

Co się dzieje: trenujesz sieć grafową GATv2, która dla każdego przystanku zbiera informacje od jego sąsiadów w grafie (pobliskich POI), **sama uczy się**, które połączenia są ważne (mechanizm uwagi), i na tej podstawie przewiduje cenę. Potem uczciwie porównujesz ją z baselinem z Fazy 2: te same dane, ten sam podział, te same metryki. **Zanim zaczniesz — rozstrzygnij poprawkę 2.1** (gdzie żyje etykieta), bo kod z planu w obecnej formie się nie spina.

Narzędzia i ich role:

- **GATv2 (Graph Attention Network v2)** — typ warstwy GNN z uwagą: każdy węzeł "ocenia", którzy sąsiedzi są dla niego istotni, zamiast traktować wszystkich po równo. To bezpośrednia

odpowiedź na główny grzech starego silnika (wagi tierów wpisane ręcznie): tutaj wagi wychodzą **z danych**. Rola: główny model AI projektu.

- **NeighborLoader** — mechanizm PyG, który zamiast ładować cały graf na kartę, wycina małe podgrafy (węzeł + próbka sąsiadów w 2 skokach) i podaje je porcjami. Rola: strażnik VRAM — bez niego 16 GB zapchałoby się błyskawicznie.
- **to_hetero** — funkcja PyG, która "rozmnaża" model homogeniczny na wszystkie typy węzłów/krawędzi grafu heterogenicznego. Rola: bez niej Twój GATv2 nie zrozumie HeteroData (patrz 2.1).
- **(opcjonalnie) PyTorch Lightning** — framework porządkujący pętlę treningową. Na MVP rekomendowana czysta pętla PyTorch (patrz Część I §3).

Słowniczek Fazy 4:

- **GNN (Graph Neural Network)** — sieć neuronowa działająca na grafie; uczy się reprezentacji węzłów, przekazując informacje między sąsiadami.
- **Message passing (przekazywanie wiadomości)** — mechanizm działania GNN: w każdej warstwie węzeł zbiera "wiadomości" od sąsiadów i aktualizuje swój stan. Po 2 warstwach "wie" o sąsiadach sąsiadów.
- **Mechanizm uwagi (attention)** — nauczalne "ważenie" sąsiadów: model sam odkrywa, że dla tego przystanku galeria waży 72%, a mikropark 0,5% — zamiast dostać te wagi od człowieka.
- **Głowa uwagi (attention head)** — jedna niezależna "perspektywa" uwagi; 4 głowy = 4 równoległe wzorce ważenia, potem sklejane. Więcej głów = większa pojemność i większy VRAM.
- **Eksplozja sąsiedztwa (neighborhood explosion)** — lawinowy wzrost liczby węzłów potrzebnych przy kolejnych skokach po grafie ($15 \text{ sąsiadów} \times 10 \text{ sąsiadów} = 150 \text{ węzłów}$ na jeden startowy). Powód istnienia NeighborLoadera.
- **Batch (paczka)** — porcja danych przetwarzana naraz (u Ciebie 128 podgrafów). Kompromis: większy batch = szybciej, ale więcej VRAM.

- **Epoka (epoch)** — jedno pełne przejście przez wszystkie dane treningowe; trening to wiele epok (plan: do 200 z early stoppingiem).
- **Funkcja straty (loss)** — liczba mówiąca "jak bardzo się mylę", którą trening minimalizuje. U Ciebie MSE.
- **MSE (Mean Squared Error)** — jak MAE, ale błędy podnosi do kwadratu, więc mocno karze duże wpadki. Standardowa strata w regresji.
- **Propagacja wsteczna (backpropagation)** — algorytm, który "cofa się" od błędu przez całą sieć i mówi każdemu parametrowi, w którą stronę się poprawić.
- **Optymalizator / Adam** — przepis, jak dokładnie robić te poprawki. Adam to rozsądny domyślny wybór.
- **Learning rate (tempo uczenia)** — wielkość pojedynczego kroku poprawki (0.001). Za duże = model skacze i nie zbiega; za małe = uczy się wieki.
- **Weight decay** — delikatne "ściąganie wag do zera" w każdym kroku; forma regularyzacji.
- **Dropout** — losowe wyłączanie części sieci w treningu (u Ciebie 30%), żeby nie uzależniła się od pojedynczych połączeń; klasyczny lek na przeuczenie przy małych danych.
- **ELU** — funkcja aktywacji (nieliniowość między warstwami); szczególnie techniczny, działa "z pudełka".
- **Konwergencja (zbieżność)** — moment, gdy strata przestaje spadać: model "doszedł". Wykres straty po epokach to Twój podstawowy monitor zdrowia treningu.
- **OOM (Out Of Memory)** — błąd "zabrakło pamięci" (VRAM/RAM). Cała choreografia NeighborLoader + batch 128 + 64 kanały istnieje po to, żeby go uniknąć.
- **Checkpoint (.pt)** — zapisany stan wytrenowanego modelu na dysku, żeby nie trenować od nowa.
- **Maska (train_mask)** — wektor prawda/fałsz mówiący, które węzły należą do treningu, a które do testu — bo w grafie nie da się fizycznie "wyciąć" testu bez zrywania krawędzi.

FAZA 5 — Interpretacja GNN ("dlaczego model tak uważa")

Co się dzieje: wyciągasz z wytrenowanego GNN wyjaśnienia: dla każdego przystanku — które **połączenia** (do których konkretnych POI) najbardziej napędzają przewidywaną cenę i które **cechy** ważą najwięcej. Produkt: GeoJSON z "TOD Premium per przystanek + top 3 połączenia", gotowy na mapę.

Narzędzia i ich role:

- `torch_geometric.explain` (**Explainer**) — wysokopoziomowe API PyG do wyjaśniania GNN. Plan słusznie koryguje `AI_research_potential.md`: biblioteka `shap` **nie działa** z sieciami grafowymi — to jest jej zamiennik.
- **GNNExplainer** — algorytm uczący się **masek**: przygasza krawędzie/cechy i patrzy, bez których predykcja się sypie. Wynik: "te połączenia były kluczowe". Odpowiada na pytanie o **krawędzie**.
- **CaptumExplainer / Integrated Gradients** — metody gradientowe z biblioteki Captum, podpięte przez to samo API. Odpowiadają raczej na pytanie o **cechy węzła**. Ważne: to **dwa różne pytania** — nie oczekuj identycznych wyników i nie traktuj rozbieżności jako błędu.

Słowniczek Fazy 5:

- **XAI (Explainable AI)** — dziedzina tłumaczenia decyzji modeli. Krytyczna, gdy wynik idzie "na stół radnego" — czarna skrzynka nie przekona nikogo.
- **Maska krawędzi / maska cech (edge mask / node mask)** — wektor wag 0–1 mówiący, jak ważna była każda krawędź/cecha dla danej predykcji. Główny produkt GNNExplainera.
- **Integrated Gradients** — metoda: prowadzi wejście od "pustego" do prawdziwego i całkuje po drodze gradienty, przypisując każdej cesze jej wkład. Solidna teoretycznie, standard w XAI.

- **Gradient** — kierunek i siła, z jaką zmiana wejścia zmienia wynik. Fundament i uczenia, i metod wyjaśniania.
 - **Ważność globalna vs lokalna** — globalna: "co jest ważne średnio w całym mieście" (to daje SHAP na XGBooście); lokalna: "co jest ważne dla TEGO przystanku" (to daje GNN + explainer). Przewaga Fazy 5 nad Fazą 2 polega właśnie na lokalności: TOD Premium przestaje być jedną liczbą dla miasta.
 - **EU AI Act** — unijne prawo o AI wymagające m.in. przejrzystości systemów w zastosowaniach wrażliwych; plan używa go jako argumentu za inwestycją w wyjaśnialność.
-

FAZA 6 — Pustynie transportowe ("gdzie popyt bije podaż")

Co się dzieje: generujesz **syntetyczny** popyt na przejazdy (model grawitacyjny: ludzie "ciążą" do celów tym słabiej, im dalej), porównujesz go z faktyczną podażą (częstotliwość z GTFS) i szukasz heksów, gdzie szacowany popyt >> podaż. **Pamiętaj o poprawce 2.6:** to jest szacunek modelowy, nie pomiar — nazywaj go tak wszędzie; oraz o rozjeździe nazw ("Deep Gravity" w nagłówku vs klasyczna grawitacja w kodzie).

Narzędzia i ich role:

- **scikit-mobility (skmob)** — biblioteka do modelowania przepływów ludzkich. Rola: dopasowanie modelu grawitacyjnego i wygenerowanie syntetycznej macierzy OD z Twojej siatki ludności i celów.
- **H3** — wspólna siatka heksagonalna, w której raportujesz pustynie ("ten heks: popyt 3× podaż, 1 200 mieszkańców, 14 min pieszo do przystanku").

Słowniczek Fazy 6:

- **Macierz OD (Origin–Destination)** — tabela "ilu ludzi podróżuje z A do B". Miasta kupują takie dane od operatorów komórkowych za

duże pieniądze; Ty generujesz przybliżenie z modelu.

- **Model grawitacyjny** — klasyczna formuła: przepływ między A i B rośnie z "masą" obu (ludność, atrakcyjność) i maleje z odległością — analogia do grawitacji Newtona. Twój stary silnik używa wariantu Huffa; tu dopasowujesz parametry do danych zamiast wpisywać je ręcznie.
- **Deep Gravity** — **neuronowa** wersja powyższego (sieć zamiast wzoru). Plan używa nazwy, ale kod robi wersję klasyczną — patrz 2.6(a). Na MVP klasyczna jest OK; po prostu nazwij ją poprawnie.
- **Doubly constrained (podwójnie ograniczony)** — wariant modelu, w którym sumy wyjazdów z każdego źródła i przyjazdów do każdego celu muszą się zgadzać z zadanymi. Bardziej realistyczny niż wariant swobodny.
- **Popyt syntetyczny** — przepływy **wygenerowane przez model**, nie zmierzone. Słowo "syntetyczny" powinno pojawiać się w każdym raporcie tej fazy.
- **Kalibracja** — dopasowanie parametrów modelu tak, by odtwarzał znane wielkości (tu: rozkład ludności/celów). Bez zewnętrznych danych o realnych przepływach kalibracja jest częściowa — kolejny powód do pokory w nazewnictwie.
- **Wskaźnik popyt/podaż (demand/supply ratio)** — Twoja definicja pustyni: im wyższy, tym gorzej obsłużony obszar względem szacowanego zapotrzebowania.

FAZA 7 — Propozycje nowych przystanków ("kandydaci, nie wyroki")

Co się dzieje: z pustyni Fazy 6 generujesz kandydatów (środek heksa → przyciągnięty do najbliższej drogi → odfiltrowany, jeśli za blisko istniejącego przystanku), wstawiasz każdego do grafu, puszczasz przez GNN **inferencję** (samo przewidywanie, bez uczenia) i rankingujesz wg złożonego wyniku. **Poprawka 2.6(b) obowiązuje tu najmocniej:** wyniki opisuj językiem współwystępowania, nie skutku.

Narzędzia i ich role:

- **OSMnx** — "snap do drogi": kandydat nie może wisieć w polu, musi leżeć przy realnej ulicy.
- **r5py** — policzenie czasów dojścia z nowego kandydata do okolicznych POI (nowe krawędzie dla grafu).
- **wytrenowany GNN (Faza 4)** — forward pass na grafie z wstawionym węzłem: "jak zmienia się przewidywana cena otoczenia".

Słowniczek Fazy 7:

- **Inferencja (inference)** — użycie **gotowego** modelu do przewidywania na nowych danych, bez dalszego trenowania. Tania obliczeniowo (jeden przebieg w przód).
 - **Forward pass** — pojedyncze "przepuszczenie" danych przez sieć od wejścia do wyniku; połowa treningu (druga połowa to backpropagation), a w inferencji — całość.
 - **Snapping** — przyciąganie punktu do najbliższego elementu sieci (tu: do segmentu drogi).
 - **Kanibalizacja** — sytuacja, w której nowy przystanek "kradnie" pasażerów istniejącemu zamiast obsłużyć nowych ludzi. Stąd filtr min. 200 m od istniejących.
 - **Composite score (wynik złożony)** — ranking z kilku składników naraz (ludność × wzrost ceny × poprawa dostępności). Uwaga metodologiczna: wagi składników znów wybiera człowiek — pokaż je jawnie i najlepiej daj suwak w dashboardzie, zamiast ukrywać w kodzie.
 - **Korelacja ≠ przyczynowość** — fundament ostrożności tej fazy: model widzi, że przystanki współwystępują z drogami okolicami; nie widzi, czy je *tworzą*. Wnioski formułuj jako "kandydat do analizy".
 - **Symulacja what-if** — eksperyment "co by było, gdyby" wykonywany w modelu. Wartościowa do eksploracji, ale jej wiarygodność jest ograniczona wiarygodnością modelu — nigdy odwrotnie.
-

FAZA 8 — Warstwa serwująca ("wyniki na mapę")

Co się dzieje: wyniki (predykcje, wyjaśnienia, pustynie, kandydaci, embeddingi) trafiają tam, gdzie ktoś je zobaczy: do dashboardu z mapą, opcjonalnie przez API i bazę wektorową do wyszukiwania "podobnych okolic". **Przypomnienie z Części I §3:** na MVP wystarczą statyczne GeoJSON-y + `sklearn`; Qdrant i FastAPI dołożą przy skali krajowej.

Narzędzia i ich role:

- **GeoJSON** — tekstowy format obiektów geograficznych z właściwościami; "waluta wymiany" między Twoim AI a każdą mapą webową. Rola: format wszystkich produktów Faz 5–7.
- **Qdrant** — baza danych **wektorowa**: przechowuje embeddingi i błyskawicznie odpowiada na pytania "znajdź k najbardziej podobnych", z filtrem geograficznym (tylko w tym poligonie). Rola: silnik funkcji "znajdź okolice jak moja" — przy dużej skali.
- **FastAPI** — biblioteka do budowy szybkiego API w Pythonie. Rola: "gniazdka" (`/api/v1/deserts/kielce ...`), przez które dashboard prosi o dane.
- **Next.js / urban-dashboard** — Twój istniejący frontend; dostaje gotowe GeoJSON-y i rysuje warstwy (heatmapa premium, czerwone heksy pustyń, markery kandydatów).

Słowniczek Fazy 8:

- **Embedding (wektor osadzeń)** — "odcisk palca" węzła wyprodukowany przez GNN: wektor np. 64 liczb streszczający jego sytuację w grafie. Kluczowa własność: **podobne miejsca mają podobne wektory** — dlatego szukanie podobnych okolic to szukanie bliskich wektorów.
- **Podobieństwo kosinusowe (cosine similarity)** — miara podobieństwa dwóch wektorów (kąt między nimi); standard przy embeddingach.
- **k-NN search** — "znajdź k najbliższych wektorów". Na małej skali: `sklearn`; na dużej: baza wektorowa.

- **Geo-filtrowanie / geo-polygon** — zawężenie wyszukiwania do obszaru na mapie ("podobne, ale tylko w Śródmieściu").
 - **API / endpoint** — interfejs, przez który programy rozmawiają / konkretny adres jednej usługi w tym interfejsie.
 - **Payload** — dodatkowe dane doczepione do wektora w bazie (nazwa przystanku, miasto, współrzędne), zwracane razem z wynikiem wyszukiwania.
 - **Upsert** — operacja "wstaw albo zaktualizuj" rekord w bazie.
 - **Warstwa (map layer)** — jeden włączany/wyłączany "plaster" mapy (heatmapa, markery, heksy).
-

CZĘŚĆ III — Słowniczek przekrojowy

Terminy, które nie należą do jednej fazy, tylko przewijają się wszędzie.

A. Rdzeń uczenia maszynowego

- **Model** — program, który uczy się wzorca z danych zamiast dostać reguły od programisty. Tu: związek "cechy okolicy → cena m²".
- **Trening** — iteracyjny proces: model zgaduje → mierzy błąd → poprawia parametry → od nowa.
- **Parametry / wagi** — liczby wewnątrz modelu, których uczy się on sam (w odróżnieniu od hiperparametrów, które ustawiasz Ty).
- **Przeuczenie (overfitting)** — model "wykuł" dane treningowe zamiast zrozumieć wzorzec; świetny na treningu, słaby na nowych danych. Główny wróg przy małych zbiorach (Kielce!). Leki: regularyzacja, dropout, early stopping, uczciwy podział.
- **Niedouczenie (underfitting)** — model za prosty; nie łapie nawet oczywistego wzorca.
- **Baseline** — najprostszy sensowny model ustalający poprzeczkę; wszystko droższe musi go pobić albo wnieść coś innego (u Ciebie: wgląd przestrzenny).

- **Walidacja krzyżowa (cross-validation)** — wiele rotacji podziału train/test zamiast jednego; daje ocenę **z niepewnością**. Rekomendowana przy małej liczbie bloków przestrzennych w Kielcach.
- **Benchmark** — uczciwe porównanie modeli na tych samych danych, tym samym podziale i tych samych metrykach. §5.4 planu to właśnie to.
- **Wynik negatywny** — rzetelnie udokumentowane "nie zadziałało i wiem dlaczego". W nauce (i dobrym portfolio) pełnoprawny rezultat.
- **Uczenie nienadzorowane** — bez etykiet; model sam szuka struktury (klastry, embeddingi). W planie: wspomniany VGAE (odłożony).
- **Klasyfikacja vs regresja** — przewidywanie kategorii vs liczby. Cały ten projekt to regresja.
- **MVP (Minimum Viable Product)** — najmniejsza wersja, która działa i udowadnia tezę. Tu: Kielce, Fazy 0–5.
- **Notebook (Jupyter)** — interaktywny dokument kod+wyniki+opis; dobry do eksploracji, gorszy do produkcji (stąd plan trzyma produkcję w skryptach `.py`).

B. Podstawy danych przestrzennych (GIS)

- **GIS** — ogólnie: praca z danymi mającymi lokalizację.
- **Geometria** — kształt obiektu: punkt (przystanek), linia (ulica), poligon (budynek, heks).
- **GeoDataFrame (gdf)** — tabela GeoPandas z kolumną geometrii.
- **Centroid** — geometryczny środek obszaru.
- **Układ współrzędnych (CRS)** — system opisu położenia. **WGS84 (EPSG:4326)** = stopnie z GPS, dobre do map, złe do metrów; **EPSG:2180** = polski układ metryczny, dobry do mierzenia odległości. Stąd podwójne kolumny (`lat/lon` i `x_2180/y_2180`).
- **Reprojekcja** — przeliczenie między układami współrzędnych.
- **Izochrona** — obszar osiągalny w X minut z punktu.

- **Tesselacja** — pokrycie mapy siatką bez dziur i nakładek (u Ciebie: heksy H3).
- **Rozdzielczość H3** — poziom szczegółowości heksów: res 9 ≈ drobne (agregacja cech), res 7 ≈ grube (bloki train/test).
- **R-drzewo (r-tree)** — indeks przestrzenny przyspieszający pytania "co jest w tym prostokącie"; wbudowany w GeoPackage.
- **Spatial join** — łączenie tabel po relacji przestrzennej ("przypisz transakcje do przystanków w 500 m").

C. Źródła danych projektu

- **GTFS** — światowy standard rozkładów jazdy (pliki CSV: linie, przystanki, godziny). Źródło `transit_freq`, liczby linii, rozpiętości godzinowej.
- **OSM (OpenStreetMap)** — otwarta, społecznościowa mapa świata; źródło budynków, ulic, POI. Słabość: nierówne uzupełnienie tagów ("szwajcarski ser").
- **Tag OSM** — para klucz=wartość opisująca obiekt (`amenity=hospital`, `building:levels=4`).
- **hstore / all_tags** — kolumna trzymająca wszystkie tagi obiektu naraz.
- **Overture Maps (OMF)** — "uporządkowany" konkurent OSM (Meta/Amazon/Microsoft/TomTom) z doszacowanymi lukami; wspierany przez city2graph, opcja na przyszłość.
- **RCN** — Rejestr Cen Nieruchomości: urzędowa baza transakcji. Twoje **ground truth** — prawdziwe ceny, na których uczy się wszystko.
- **Ground truth** — "prawda odniesienia", z którą porównujesz predykcje.
- **GUGiK** — Główny Urząd Geodezji i Kartografii (wystawca RCN).
- **GUS / NSP 2021** — statystyka publiczna / spis powszechny; źródło siatki ludności 250 m.
- **WFS** — protokół pobierania danych przestrzennych z serwerów (tak ściągasz RCN). Bywa niestabilny — stąd cała saga skryptów

naprawczych 08/09 i flag `repair_source`.

- **GML / XLink** — format XML danych geograficznych / mechanizm odwołań między rekordami; zerwane XLinki to typowa awaria WFS-ów powiatowych.
- **TERYT** — krajowy system kodów jednostek terytorialnych; klucz Twojego cache'a.
- **Geofabrik** — serwis z gotowymi wycinkami OSM w PBF.

D. Pojęcia z Twojego starego silnika (v13.0) — żebyś rozumiał, co AI zastępuje

- **TOD (Transit-Oriented Development)** — planowanie miasta wokół transportu publicznego; teza, że dobra komunikacja podnosi wartość okolicy.
- **TOD Premium** — wyrażona w złotych "wartość transportu w cenie mieszkania". Flagowy produkt projektu (z receptą poprawioną w 2.3).
- **Model Huffa** — wariant modelu grawitacyjnego: prawdopodobieństwo wyboru celu maleje wykładniczo z odległością ($pull = \exp(-K \cdot dist)$). Stary silnik liczy go ze stałymi wpisanymi ręcznie — to dokładnie ten typ założenia, który AI ma zastąpić uczeniem.
- **Tier (T0–T6)** — ręczna hierarchia ważności POI (lotnisko: 5 mln pkt ... park: 100 pkt). Główny zarzut `AI_research_potential.md`: to ludzki bias zabetonowany w danych. Dlatego eksport `15b` jest **bez** tierów.
- **Bias (uprzedzenie)** — założenie człowieka wstrzyknięte w dane/model, które blokuje odkrycie prawdziwych zależności.
- **Entropy Shannona** — miara różnorodności funkcji w okolicy (mixed-use); wysoka = "tętniące życiem" pomieszczenie sklepów, biur, mieszkań.
- **Z-Score** — standaryzacja: "ile odchyień standardowych od średniej". Stary silnik skleja nim cechy w jeden wynik ze sztywnymi

wagami (0.35/0.35/0.15/0.15) — AI dostaje cechy surowe i wagi ustala samo.

- **log1p** — logarytm(1+x); spłaszcza rozkłady z długim ogonem (kilka gigantycznych wartości) przed standaryzacją.
- **IQR** — rozstęp międzykwartylowy (środkowe 50% danych); filtr wartości absurdalnych przy medianie cen.
- **MAD (Median Absolute Deviation)** — odporna miara rozrzutu; reguła "6-sigma MAD" łapie ekstremalne błędy, np. "teleportujące się" stacje PKP (błędne współrzędne w danych źródłowych).
- **Outlier** — wartość drastycznie odstająca. Dylemat: błąd do usunięcia czy rzadki, cenny sygnał? Plan słusznie ostrzega przed nadgorliwym "pomaganiem" modelowi przez czyszczenie.
- **Spatial Dissolve** — sklejanie rozczłonkowanych kompleksów OSM (szpital z 15 pawilonów) w jeden obiekt, żeby nie liczyć go 15×.
- **Agglomerative Clustering** — algorytm łączenia bliskich punktów w grupy (progi 150 m/100 m w kroku 15). Krytyka: niszczy wariancję topologii, którą GNN mógłby wykorzystać — dlatego dane dla AI idą **bez** tego sklejanja.
- **Heurystyka** — reguła "z ręki" działająca zwykle dobrze, ale bez gwarancji i bez uczenia. Cały silnik v13.0 to system heurystyczny — świetny kalkulator, nie AI. Sedno projektu: sprawdzić, czy uczenie z danych zrobi to lepiej lub odkryje coś, czego heurystyka nie widzi.

E. Sprzęt i inżynieria

- **RAM vs VRAM** — pamięć operacyjna komputera (32 GB, tu żyje r5py i GeoPandas) vs pamięć karty graficznej (16 GB, tu żyje trening GNN). To **dwa osobne budżety** — plan pilnuje obu
(`set_max_memory("12G")` , NeighborLoader).
- **psutil** — biblioteka do pomiaru zużycia RAM/CPU; plan używa jej w kryteriach akceptacji ("peak RAM < 20 GB").
- **Pipeline (potok)** — łańcuch kroków przetwarzania danych; Twoje kroki 00–15.
- **ETL (Extract–Transform–Load)** — wzorzec: pobierz → przekształć → zapisz. Twój system to "Spatial ETL".

- **Orkiestrator** — program sterujący kolejnością kroków (`orchestrator.py`); AI wpinasz jako opcjonalną flagę `--ai-export` , nie ruszając rdzenia.
- **Idempotentność / wznowialność** — właściwość pipeline'u: ponowne uruchomienie nie psuje wyników, przerwany krok da się wznowić.
- **Cache** — zapamiętane wyniki kosztownych operacji (Twój `SmartCache` po TERYT-ach), żeby nie pobierać/liczyć drugi raz.
- **Tensor** — wielowymiarowa tablica liczb; podstawowa jednostka danych PyTorch'a.
- **DuckDB** — lekka, szybka baza analityczna "w pliku"; city2graph używa jej pod spodem do GTFS.
- **Determinizm / `random_state=42`** — przypięcie ziarna losowości, żeby wyniki były powtarzalne między uruchomieniami. Obowiązkowe w badaniach.
- **Root Cause Analysis (RCA)** — szukanie **źródłowej** przyczyny błędu zamiast łatania objawów; oś sporu wokół skryptów 08/09 (plan wybiera kompromis: naprawiaj, ale flaguj — rozsądnie).
- **Halucynacja (AI)** — pewnie brzmiąca, zmyślona informacja od modelu językowego. W tym projekcie spotkałeś je już dwa razy: nieistniejące API city2graph i wątpliwe cytowania w `AI_research_potential.md` . Antidotum: weryfikacja z dokumentacją i źródłami.

Trzy zasady na ścianę (wersja Fable 5)

1. **Kod z planów AI to pseudokod, a wewnętrzna spójność planu też wymaga audytu.** Największe pułapki tego planu to nie brakujące pomysły, tylko rozjazdy między jego własnymi sekcjami (graf heterogeniczny vs model homogeniczny; "Deep Gravity" vs zwykła grawitacja).
2. **Flagowa liczba wymaga flagowej staranności.** "TOD Premium za +1 kurs/h" licz kontryfaktykiem/ALE, nie medianą SHAP — bo to jedyna liczba z całego projektu, którą ktoś zacytuje.

3. **Mocny baseline + uczciwy język = Twoja przewaga.** Jeśli LightGBM pobije GNN — to wynik. Jeśli GNN pokaże bariery przestrzenne, których tabela nie widzi — to też wynik. Przegrywasz tylko wtedy, gdy nazwiesz korelację przyczyną.